

Análisis de etapas de compilación de programa en C en Arquitectura x-86

Fernando Vicente Muñoz
Departamento de Sistemas Electrónicos
(Benemérita Universidad Autónoma de
Aguascalientes (UAA)
Aguascalientes, México.
al575994@edu.uaa.mx

Abstract— En este reporte se analizan las diferentes etapas involucradas en la compilación de un programa escrito en lenguaje C para la arquitectura x86-64, utilizando como ejemplo un programa encargado de generar la sucesión de Fibonacci. Se estudian las etapas de preprocesamiento, compilación, ensamblado y enlazado mediante los archivos intermedios generados con GCC. Además, se relaciona el código ensamblador obtenido con la arquitectura de conjunto de instrucciones (ISA) de x86-64, identificando algunos registros e instrucciones presentes en el programa. Este análisis permite comprender de manera práctica cómo un código escrito en un lenguaje de alto nivel atraviesa diferentes transformaciones hasta obtener una representación que puede ser ejecutada por el procesador[8]

Keywords— preprocesamiento, compilación, ensamblado, enlazado, arquitectura x86-64, lenguaje C, GCC, ISA.

I. INTRODUCCIÓN

A diferencia de los seres humanos, las computadoras no comprenden lenguajes de la misma manera que nosotros. En su nivel más básico, un procesador trabaja con instrucciones representadas mediante valores binarios, es decir, combinaciones de ceros y unos que corresponden a distintos estados eléctricos y que siguen los principios de la lógica booleana. Aunque sería posible expresar directamente las instrucciones de esta manera, programar utilizando únicamente código binario resultaría una tarea sumamente extensa, complicada y propensa a errores.

Para facilitar la comunicación entre el programador y la computadora surgieron los lenguajes de programación de alto nivel, los cuales permiten expresar instrucciones mediante estructuras mucho más cercanas y comprensibles para las personas. Sin embargo, estas instrucciones no pueden ser ejecutadas directamente por el procesador, por lo que necesitan pasar por un proceso de traducción. En los lenguajes compilados, como C, esta tarea se realiza mediante distintas etapas que transforman progresivamente el código fuente hasta obtener instrucciones en lenguaje máquina que pueden ser ejecutadas por la arquitectura para la cual fueron generadas. En un sistema de compilación como GCC, esta transformación puede comprender las etapas de preprocesamiento, compilación, ensamblado y enlazado [1], [2].

En este reporte se analizan las diferentes etapas involucradas en la compilación de un programa escrito en lenguaje C para una arquitectura x86, utilizando como ejemplo un programa encargado de generar los primeros N números de la serie de Fibonacci. A partir de este ejemplo se observa cómo el código original atraviesa las etapas de

preprocesamiento, compilación, ensamblado y enlazado hasta obtener finalmente un archivo ejecutable.

II. Diseñando un programa en C

Para poder compilar un programa necesitamos contar con tres elementos principales: un lenguaje de programación, un código funcional y un compilador. En este caso se eligió el lenguaje C, ya que permite desarrollar programas utilizando instrucciones de alto nivel que posteriormente pueden ser traducidas para su ejecución en la arquitectura objetivo.

Como punto de partida para observar este proceso, se diseñó un programa capaz de generar los primeros N números dados por el usuario de la sucesión de Fibonacci.

El funcionamiento del programa consiste en solicitar al usuario la cantidad de términos que desea obtener. A partir de los valores iniciales de la sucesión, 0 y 1, cada nuevo término se calcula mediante la suma de los 2 anteriores, repitiendo este procedimiento hasta alcanzar la cantidad indicada. De esta manera, se obtiene un código funcional que servirá como punto de partida para analizar las transformaciones que ocurren durante las diferentes etapas de compilación, el programa es el siguiente:

```
GNU nano 7.2 fibonacci.c
//Programa que genera la sucesión de Fibonacci
Fernando Vicente Muñoz ISC 3C, Arquitectura de computadores
Actividad 17/

#include <stdio.h>

int main(void) {
    int actual = 0, sig = 1, suma = 0, num_limite = 0; //Enteros que utilizaremos para el programa

    printf("¿Hasta que número de término quieres llegar: ");
    scanf("%d",&num_limite);

    if(num_limite < 1) {
        printf("No se vale valores menores a uno.\n");
        return 1;
    }

    for (int i = 0; i <= num_limite; i++) {
        printf("Número %d: %d\n", i, actual);
        suma = actual + sig; //Guardamos en variable suma para poder sustituir actual por sig sin perder
        actual = sig;
        sig = suma;
    }

    printf("Se ha terminado el programa exitosamente!!!\n");
    return 0;
}
```

Fig 1. Código fuente del programa para generar la sucesión de Fibonacci en lenguaje C.

Como se puede observar en el código, el programa mantiene una interacción con el usuario al solicitar la cantidad de términos de la serie de Fibonacci que se desean obtener. Sin embargo, más allá de esta interacción, lo que resulta de mayor interés para este análisis es la lógica que existe detrás del algoritmo. El programa sigue una ejecución secuencial, en la que cada instrucción se procesa siguiendo un orden determinado y las operaciones dependen de los resultados obtenidos previamente. Esto permite observar de manera sencilla algunas de las características del lenguaje C, como el

uso de variables, estructuras de control, operaciones aritméticas y llamadas a funciones. A partir de este código es posible analizar cómo estas instrucciones, inicialmente comprensibles para el programador, se transforman progresivamente durante las diferentes etapas de compilación hasta llegar a una representación que pueda ser ejecutada por la máquina.

III. Etapas del proceso de compilación

Una vez desarrollado y comprobado el funcionamiento del programa en C, el siguiente paso consiste en transformar el código fuente en un programa que pueda ser ejecutado por la computadora. Aunque normalmente este proceso puede realizarse mediante una sola instrucción, internamente GCC puede llevar a cabo hasta cuatro etapas principales: preprocesamiento, compilación, ensamblado y enlazado, realizadas en ese orden [1].

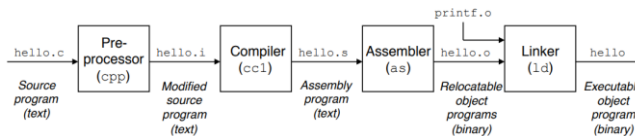


Fig. 2. Etapas del sistema de compilación de un programa en C. Tomado de [3].

Cada una de estas etapas realiza una transformación diferente sobre el programa y produce información que será utilizada posteriormente para obtener el archivo ejecutable. Para comprender este proceso, se analizarán individualmente las transformaciones que experimenta el programa de la serie de Fibonacci.

Antes de que el código sea procesado propiamente por el compilador, pasa por una etapa conocida como preprocesamiento. En ella se identifican y procesan las directivas del preprocesador, las cuales pueden reconocerse porque comienzan con el símbolo #. Entre ellas se encuentran instrucciones como #include, #define y las directivas de compilación condicional.

En nuestro programa, se puede observar desde el inicio la siguiente instrucción:

```
#include <stdio.h>
```

Esta línea indica que se requiere el contenido correspondiente al encabezado stdio.h, necesario para utilizar funciones de entrada y salida como printf() y scanf(). Durante el preprocesamiento, esta directiva es procesada antes de que el código continúe hacia las siguientes etapas de compilación.

Para observar directamente el resultado de esta primera transformación puede utilizarse GCC con la opción -E. De acuerdo con la documentación de GCC, la extensión .i identifica código fuente en C que ya ha sido preprocesado [1]:

```
gcc -E fibonacci.c -o fibonacci.i
```

Como resultado se genera el archivo fibonacci.i, que contiene el código después de haber pasado por el preprocesador. Al abrirlo, una de las primeras diferencias que puede observarse es que su extensión y cantidad de contenido son considerablemente mayores que las del archivo fibonacci.c original, debido al procesamiento de las directivas presentes en el código, en este caso con el #include estaríamos importando todo lo relacionado a la librería stdio.h, la cual es la que nos permite manejar valores de entrada y salida.

Una vez finalizado el preprocesamiento, el archivo resultante pasa a la etapa de **compilación**. En esta fase, el código escrito originalmente en un lenguaje de alto nivel comienza a transformarse en una representación mucho más cercana a las instrucciones de la arquitectura objetivo. En GCC, es posible detener el proceso después de esta etapa mediante la opción -S, obteniendo como resultado un archivo que contiene código en lenguaje ensamblador [1].

Para nuestro programa, el archivo fibonacci.i obtenido anteriormente se utilizó como entrada para la siguiente instrucción:

```
gcc -S fibonacci.i -o fibonacci.s
```

El resultado de este comando es el archivo fibonacci.s. La extensión .s es reconocida por GCC como código en lenguaje ensamblador [1].

Esta transformación resulta especialmente interesante para el análisis, ya que permite observar cómo las estructuras que anteriormente resultaban sencillas de interpretar en C comienzan a expresarse mediante instrucciones de menor nivel. Las variables, operaciones aritméticas, comparaciones y ciclos utilizados para generar la sucesión de Fibonacci ahora se encuentran representados mediante operaciones propias del lenguaje ensamblador.

```

.cfi_startproc
endbr64
pushq  %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq   %rsp, %rbp
.cfi_def_cfa_register 6
subq   $32, %rsp
movq   %fs:40, %rax
movq   %rax, -8(%rbp)
xorl   %eax, %eax
movl   $0, -24(%rbp)
movl   $1, -20(%rbp)
movl   $0, -12(%rbp)
movl   $0, -28(%rbp)
leaq   .LC0(%rip), %rax
movq   %rax, %rdi
movl   $0, %eax
call   printf@PLT
leaq   -28(%rbp), %rax
movq   %rax, %rsi
leaq   .LC1(%rip), %rax
movq   %rax, %rdi

```

Fig 3. Código Fibonacci en lenguaje ensamblador

Al comparar este resultado con el código original puede apreciarse una reducción en el nivel de abstracción. Mientras que en C una operación puede expresarse mediante una sola instrucción, en ensamblador puede requerir diferentes operaciones relacionadas con registros, memoria, comparaciones y saltos. Este archivo todavía mantiene una representación que puede ser interpretada por una persona familiarizada con ensamblador, pero se encuentra preparado para pasar a la siguiente etapa del proceso.

Después de obtener el archivo en lenguaje ensamblador, comienza la etapa de **ensamblado**. Durante este proceso, las instrucciones contenidas en fibonacci.s son traducidas a una representación numérica y almacenadas en un archivo objeto. La documentación oficial de GNU Assembler señala que el archivo objeto contiene el programa ensamblado, además de información que posteriormente permite al enlazador integrarlo dentro de un archivo ejecutable [4].

Para generar el archivo objeto correspondiente al programa de Fibonacci se utilizó:

```
gcc -c fibonacci.s -o fibonacci.o
```

La opción -c permite compilar o ensamblar el archivo de entrada sin realizar todavía la etapa de enlazado. Como resultado, GCC produce un archivo objeto, cuya extensión convencional es .o [1]

A diferencia de los archivos anteriores, fibonacci.o ya no contiene instrucciones presentadas de una forma pensada para ser leída directamente como código fuente. El archivo objeto contiene el código ensamblado y la información necesaria para que posteriormente pueda ser procesado por el enlazador [4].

Sin embargo, obtener fibonacci.o todavía no significa que el proceso haya terminado. El archivo objeto está diseñado para servir como entrada al enlazador y puede contener información y referencias que deben resolverse antes de producir el programa ejecutable final [4].



Fig 4. Visualización del archivo objetivo fibonacci.o mediante un editor de texto

Al intentar visualizar directamente el archivo objeto, como se muestra en la Fig. 4, puede observarse que su contenido ya no presenta la estructura legible que tenían los archivos .c y .s. En su lugar aparecen principalmente caracteres sin una interpretación directa para el usuario, debido a que se trata de un archivo binario. Sin embargo, todavía es posible reconocer algunas cadenas de texto almacenadas en el programa, como el mensaje “Se ha terminado el programa exitosamente!!!”.

Para analizar el contenido del archivo objeto de una manera más comprensible se utilizó la herramienta objdump con la opción -d, la cual permite desensamblar las secciones que contienen instrucciones de máquina.

```
objdump -d fibonacci.o
```

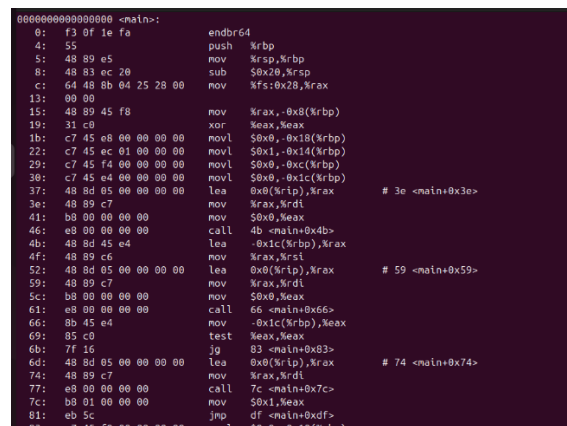


Fig 5. Desensamblado del archivo objeto fibonacci.o mediante objdump -d.

La Fig. 5 permite observar la relación entre el código máquina y su representación en lenguaje ensamblador. En la columna izquierda aparecen las direcciones u offsets de las instrucciones; posteriormente se muestran los bytes que codifican cada instrucción y, a la derecha, su representación en ensamblador.

De esta manera, el desensamblado permite comprobar que las instrucciones que anteriormente observamos en fibonacci.s ya cuentan con una representación mediante bytes dentro del archivo objeto. El programa, sin embargo, todavía debe pasar por la etapa de enlazado antes de obtener el archivo ejecutable final [4].

Una vez generado el archivo objeto fibonacci.o, todavía es necesario realizar una última etapa antes de poder ejecutar el programa: el **enlazado** o *linking*. El enlazador se encarga de combinar los archivos objeto y las bibliotecas necesarias, además de resolver las referencias a símbolos que aún se encuentran pendientes, para construir finalmente el archivo ejecutable [3], [5].

En el programa desarrollado existen funciones como printf() y scanf() cuya implementación no se encuentra escrita directamente dentro de fibonacci.c. Durante las etapas anteriores se conservaron referencias a estas funciones, las cuales deben ser resueltas como parte del proceso de construcción del programa final.

Para realizar el enlazado se utilizó el siguiente comando:

```
gcc fibonacci.o -o fibonacci
```

En este caso, fibonacci.o corresponde al archivo objeto generado en la etapa anterior, mientras que la opción -o fibonacci establece el nombre del archivo de salida. Al no utilizar opciones como -E, -S o -c, GCC continúa el proceso hasta realizar el enlazado y producir el ejecutable [1].

Al finalizar esta etapa se obtiene el archivo fibonacci, completándose así las transformaciones que comenzaron a partir del código fuente.

El enlazado representa entonces el punto en el que los diferentes elementos necesarios para el funcionamiento del programa quedan integrados en un archivo que puede ser cargado para su ejecución. De esta manera, las instrucciones que inicialmente fueron escritas mediante un lenguaje de alto nivel han atravesado todas las etapas necesarias para obtener un programa ejecutable [3], [5].

IV. ARQUITECTURA x86 Y SU CONJUNTO DE INSTRUCCIONES

Cada arquitectura de procesador se encuentra definida, entre otros elementos, por una **arquitectura de conjunto de instrucciones**, conocida como **ISA** (*Instruction Set Architecture*). Esta establece la interfaz mediante la cual el software puede comunicarse con el procesador, definiendo las instrucciones y recursos que este puede reconocer y ejecutar. Por esta razón, el código máquina generado durante la compilación depende de la arquitectura para la cual se está construyendo el programa [6]. Intel describe x86 como una familia basada en una ISA que ha evolucionado desde el procesador Intel 8086 y que actualmente continúa en las arquitecturas IA-32 e Intel 64.

En el caso de esta práctica, el código ensamblador obtenido permite observar instrucciones pertenecientes a la arquitectura **x86-64**, identificable, entre otros aspectos, por la utilización de registros de 64 bits como `%rax`, `%rbp`, `%rsp` y `%rdi`. El *Intel 64 and IA-32 Architectures Software Developer's Manual* constituye la documentación de referencia de Intel para esta arquitectura; su volumen 1 describe la arquitectura y el entorno de programación, mientras que el volumen 2 contiene la referencia completa de su conjunto de instrucciones [7].

Dentro del archivo `fibonacci.s` generado durante la práctica pueden encontrarse diferentes ejemplos de estas instrucciones. Una de las más frecuentes es `MOV`, utilizada para transferir datos entre registros o entre registros y memoria. En nuestro programa aparecen instrucciones como `movq` y `movl`, estas instrucciones permiten observar cómo operaciones que en C se expresan mediante variables y asignaciones terminan convirtiéndose en movimientos de información entre los diferentes recursos disponibles para el procesador. El conjunto de instrucciones de Intel incluye precisamente instrucciones para transferencia de datos, operaciones aritméticas, operaciones lógicas y control del flujo del programa [7].

También encontramos instrucciones relacionadas con el manejo de la pila, como `pushq` y `subq`.

Otro ejemplo particularmente importante para el programa de Fibonacci son las instrucciones aritméticas. Intel incluye dentro de este grupo instrucciones como `ADD`, `SUB`, `INC`, `DEC` y `CMP`; específicamente, `ADD` realiza una suma de enteros [7]. Esto resulta directamente relacionado con la lógica utilizada para obtener la sucesión de Fibonacci, ya que su operación fundamental consiste en sumar los dos valores anteriores para producir el siguiente.

El conjunto de instrucciones x86 también dispone de instrucciones para modificar el flujo de ejecución. Entre ellas se encuentran `JMP`, los saltos condicionales `Jcc`, `CALL` y `RET` [7]. Estas permiten representar a un nivel inferior estructuras que en C escribimos mediante ciclos, condiciones y llamadas a funciones. Por ejemplo, en el código generado durante la práctica aparecen instrucciones de salto y llamadas asociadas con el funcionamiento del programa y con funciones utilizadas desde C.

Un ejemplo visible en nuestro archivo es:

```
call printf@PLT
```

Aunque en C solamente escribimos una llamada a `printf()`, al observar el código generado podemos apreciar que esta operación termina involucrando instrucciones de control de flujo a un nivel inferior.

De esta manera, el análisis del código generado permite comprender que el compilador no produce instrucciones universales. Las instrucciones finales deben pertenecer al ISA reconocido por la arquitectura objetivo. En nuestro caso, elementos relativamente sencillos del programa en C terminan representándose mediante instrucciones de la familia x86-64, demostrando la relación existente entre el lenguaje de alto nivel, el compilador y la arquitectura sobre la cual finalmente se ejecutará el programa.

V. CONCLUSIONES

A lo largo de este trabajo fue posible comprender que la compilación de un programa no consiste simplemente en convertir un archivo escrito en C directamente en un ejecutable. Detrás de este proceso existen diferentes etapas que transforman poco a poco el código, comenzando con el preprocesamiento, continuando con la compilación y el ensamblado, hasta finalizar con el enlazado. Poder observar los archivos generados en cada una de estas etapas permitió entender de una manera más clara qué ocurre detrás de un proceso que normalmente realizamos utilizando solamente un comando.

El programa de Fibonacci también permitió relacionar estos conceptos con la arquitectura de la computadora. Al observar el código ensamblador generado se identificaron instrucciones y registros pertenecientes a x86-64, haciendo más evidente la relación que existe entre un lenguaje de alto nivel, el compilador y la ISA que finalmente entiende el procesador. De esta manera, la práctica no solo permitió conocer las etapas de compilación, sino también comprender mejor cómo las instrucciones que escribimos como programadores atraviesan diferentes representaciones hasta convertirse en instrucciones que pueden ser ejecutadas por la máquina.

REFERENCIAS

- [1] Free Software Foundation, "Options controlling the kind of output," *Using the GNU Compiler Collection (GCC)*, GNU Project. [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html>. [Accessed: Aug. 29, 2026].
- [2] Free Software Foundation, "Include syntax," *The C Preprocessor*, GNU Project. [Online]. Available: <https://gcc.gnu.org/onlinedocs/cpp/Include-Syntax.html>. [Accessed: Aug. 29, 2026].
- [3] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Boston, MA, USA: Pearson, 2016. [Online]. Available: <https://csapp.cs.cmu.edu/3e/ics3/intro/compilation.pdf>. [Accessed: Aug. 29, 2026].

- [4] Free Software Foundation, "Output (Object) File," *Using as: The GNU Assembler*, GNU Binutils. [Online]. Available: <https://sourceware.org/binutils/docs/as/Object.html>. [Accessed: Aug. 29, 2026].
- [5] Free Software Foundation, "Overview," *LD: The GNU Linker*, GNU Binutils. [Online]. Available: <https://sourceware.org/binutils/docs/ld/Overview.html>. [Accessed: Aug. 29, 2026].
- [6] Intel Corporation, "What is x86 architecture? A primer to the foundation of modern computing," *Intel Newsroom*. [Online]. Available: <https://newsroom.intel.com/tech101/x86-architecture-foundation-of-modern-computing>. [Accessed: Aug. 29, 2026].
- [7] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual*. Santa Clara, CA, USA: Intel Corporation. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. [Accessed: Aug. 29, 2026].
- [8] OpenAI, *ChatGPT*, respuesta generada al prompt "Dame el abstract en español", Aug. 29, 2026.